# Lecture VI - Minimizing Split Orders

Applied Optimization with Julia

Dr. Tobias Vlćek



## Introduction

### E-Commerce Trends

**Question:** What are current trends in e-commerce?

### E-Commerce Sales

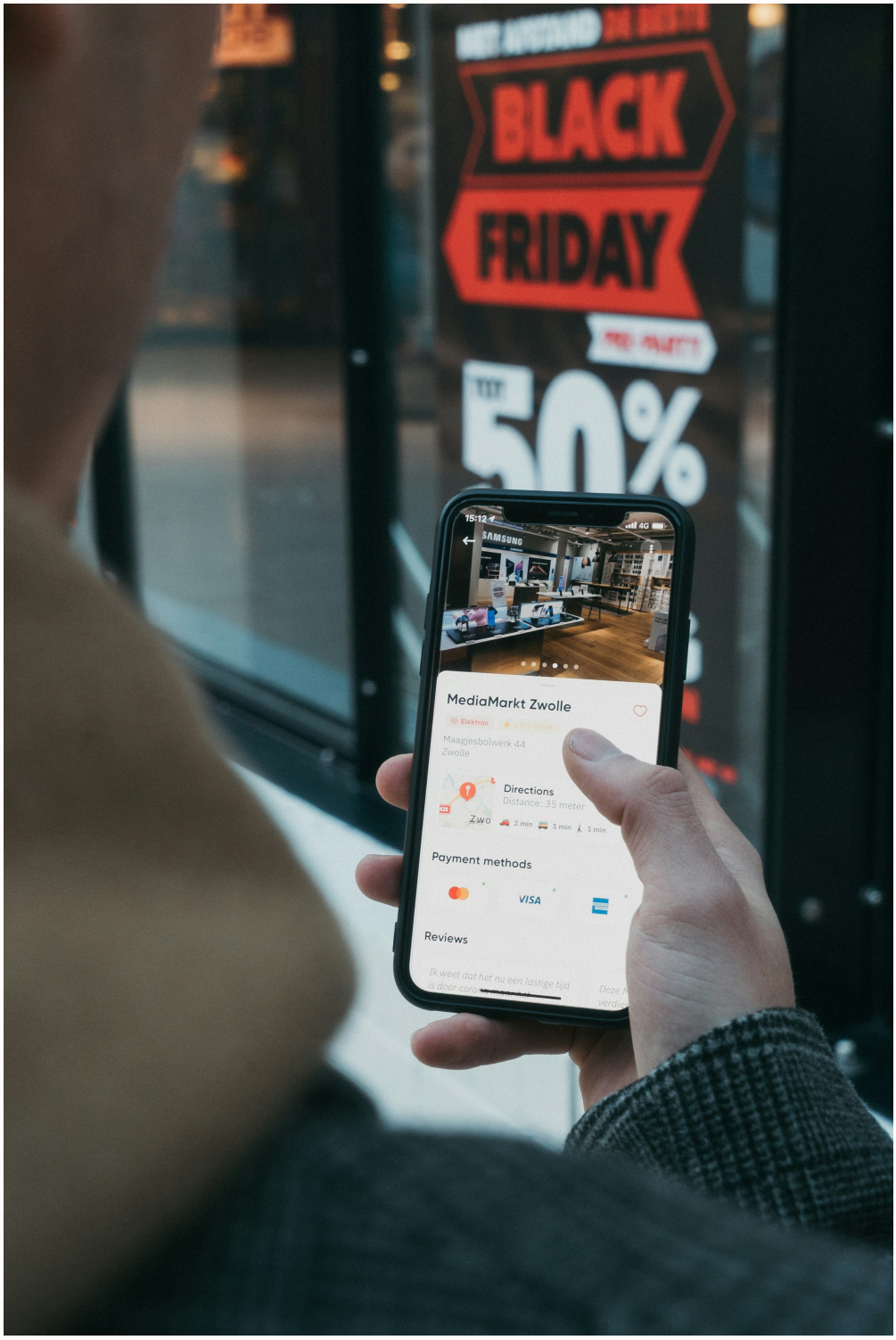
- E-Commerce sales **are growing fast**:
  - Products are **no longer bound between borders**
  - Product variety is **rising**
  - Consumer shopping patterns are **shifting**
  - Brick-and-mortar stores **lose customers to the internet**
  - Covid-19 **accelerated this trend even more**

### Parcels Worldwide

- The number of parcels is rising:
  - **2014**: 44 billion parcels [1]
  - **2019**: 103 billion parcels [2]
  - **2026**: forecast of 220 – 262 billion[1] [3]

[1] Forecast made in 2020, not an actual number

## Pressure on infrastructure




- Consumers nowadays expect **free, fast deliveries and returns**
- Existing warehouses have to store an **increasing range of products**
- Better customer service requires **faster deliveries**
- Incurred fulfillment costs **depend on the number of parcels**

## Pressure on the environment




- **Each parcel packaging** consumes resources during production
- Every dispatched parcel to the customer **causes $CO_2$ emissions**
- In case of returns, **more parcels cause more emissions**

## Learning Objectives

After this lecture, you will be able to:

- **Explain** split orders and why they matter in e-commerce

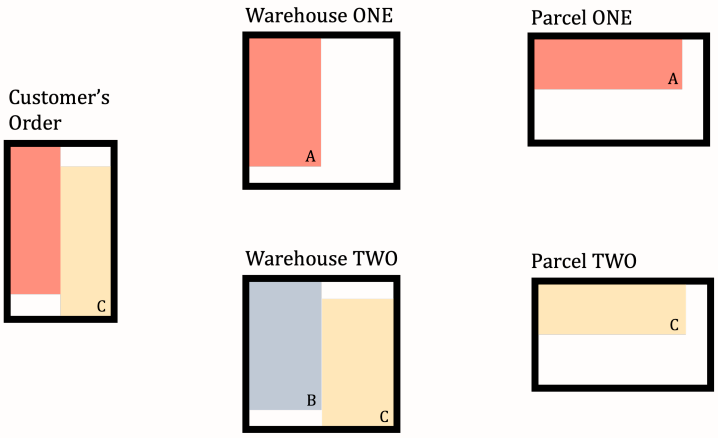
- **Build** a coappearance matrix from transactional data
- **Formulate and solve** the Quadratic Multiple Knapsack Problem (QMKP) in JuMP
- **Judge** when exact solvers reach their limits and heuristics are needed

# Problem Structure

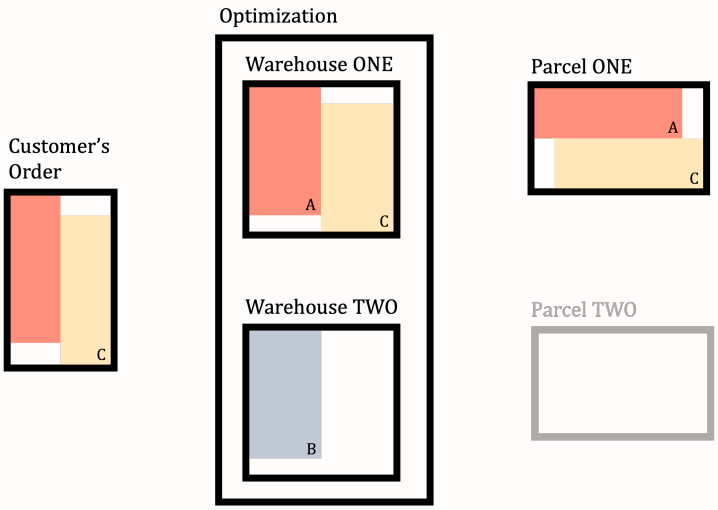
## One Order, Two Parcels

**Question:** Why did the shop send two?

### Split Order




### No Split Order




### Reason for Split Orders

**Question: Why might they occur?**

- **Stock availability**: Some products are **out of stock** at a warehouse and need to be fulfilled from another warehouse
- **Capacity constraints**: Some products are stored at **different warehouses** and need to be shipped from elsewhere

## Impact of Split Orders

**Question: What are the consequences?**

- **Higher** shipping costs
- **Increased** packaging material
- **More** $CO_2$ emissions
- **Higher operational complexity**
- **Lower** customer satisfaction

## Mitigations?

**Question: What are possible mitigations?**

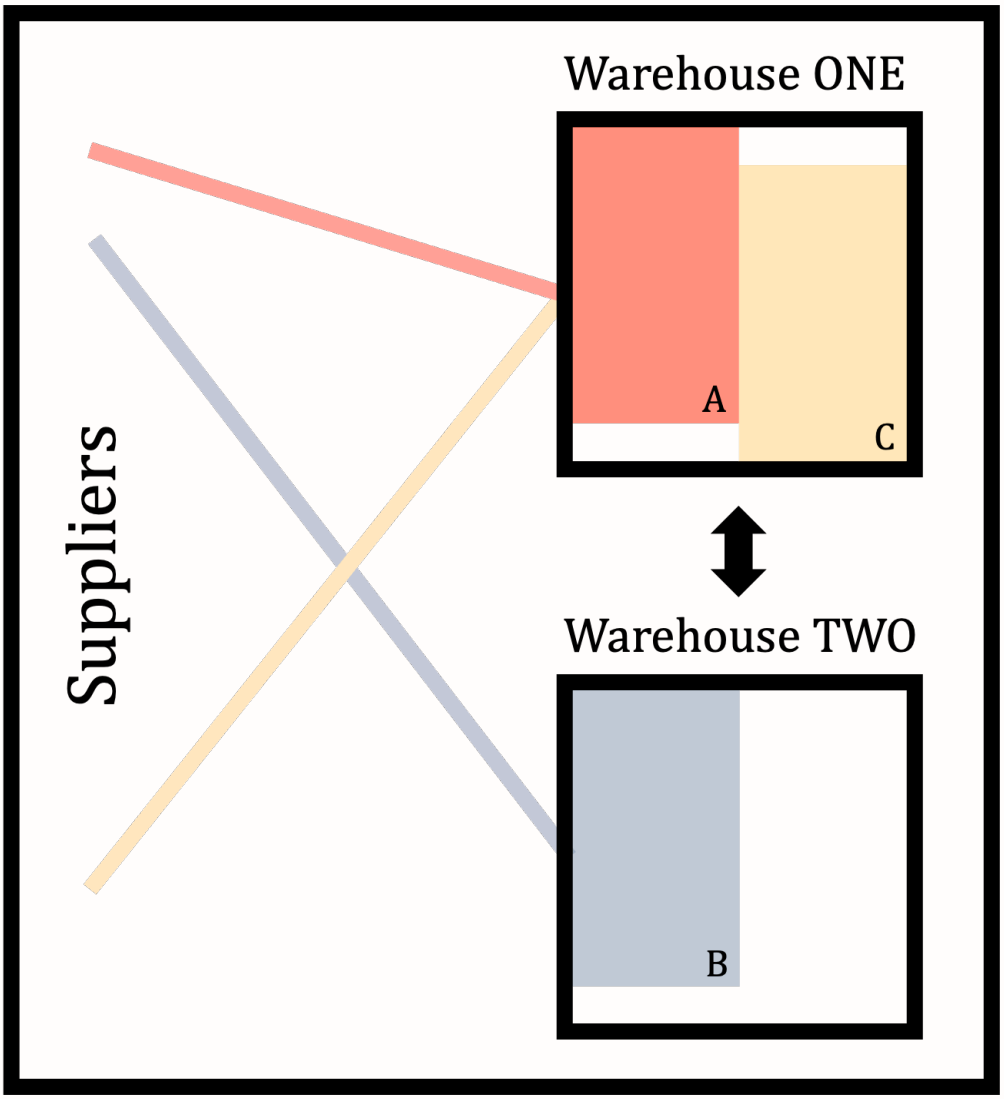
- **Consolidation**: Ship to a central warehouse before dispatch
- **Cross-docking**: Transfer goods from inbound to outbound transport with little or no storage in between
- **Transshipment**: Ship between warehouses before delivery
- **Co-allocation**: Predict **coappearance of products** and allocate them to the **same warehouse**

## Your Turn

Play: https://beyondsimulations.github.io/Applied-Optimization/games/warehouses.html

# Structure - Version 1

## Optimizing Co-allocation




**Question: What could be our objective?**

We aim to improve the **SKU[2]-warehouse allocation** to minimize the number of split parcels resulting from **SKUs being stored in different warehouses**.

## Available Sets

**Question: What could be the sets here?**

- $\mathcal{I}$ - Set of products indexed by $i \in \{1, 2, ..., |\mathcal{I}|\}$
- $\mathcal{K}$ - Set of warehouses indexed by $k \in \{1, ..., |\mathcal{K}|\}$
- $\mathcal{M}$ - Set of customer orders $m \in \{1, 2, ..., |\mathcal{M}|\}$

[2] SKU: Stock Keeping Unit

## Available Parameters

**Question: What are possible parameters?**

- $c_k$ - Storage space of warehouse $k \in \{1, ..., |\mathcal{K}|\}$, measured in number of SKUs
- $\boldsymbol{T} = (t_{m,i})$ - Past customer orders for SKUs

**Question: What could the transactional data look like?**

## Transactional Data

| $t_{m,i}$ | A | B | C | D |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 0 |
| 2 | 1 | 1 | 1 | 0 |
| 3 | 1 | 1 | 0 | 0 |
| 4 | 1 | 0 | 0 | 1 |
| 5 | 1 | 0 | 0 | 1 |
| 6 | 1 | 0 | 0 | 1 |
| 7 | 1 | 0 | 0 | 1 |
| 8 | 0 | 0 | 1 | 1 |

## Past vs. Future

- The **transactional data** $\boldsymbol{T}$ is based on **past orders**
- It is a **binary matrix** of customer orders and SKUs
- We use this data to **assume** future co-occurrence
  - **Past co-occurrence predicts future co-occurrence**

**Question: What is your opinion on the assumption?**

## Split-Order Minimization

**Question: What could be our decision variable/s?**

> **We have the following sets:**
> - $\mathcal{I}$ - Set of products indexed by $i \in \{1, 2, ..., |\mathcal{I}|\}$
> - $\mathcal{K}$ - Set of warehouses indexed by $k \in \{1, ..., |\mathcal{K}|\}$
> - $\mathcal{M}$ - Set of customer orders $m \in \{1, 2, ..., |\mathcal{M}|\}$

- $X_{i,k}$ - 1, if $i \in \mathcal{I}$ is stored in $k \in \mathcal{K}$, 0 otherwise
- $Y_{m,i,k}$ - 1, if SKU $i \in \mathcal{I}$ is shipped from warehouse $k \in \mathcal{K}$ for customer order $m \in \mathcal{M}$, 0 otherwise

### Integer Programming Model

- A. Catalán and M. Fisher [4] created an **integer model**
- Number of SKUs of E-Commerce retailers can easily be **between 10,000 - 100,000**
- The number of customer orders needed for "stable" results is one order of magnitude higher: **100,000 - 10,000,000**

**Question: Does anyone have an idea what this could mean?**

### Why is Version 1 so hard?

- Objective: **minimize the number of warehouses** that ship parcels for each customer order
- Requires $Y_{m,i,k}$: one variable per order, SKU **and** warehouse
- $\rightarrow |\mathcal{M}| \times |\mathcal{I}| \times |\mathcal{K}|$ binary variables

> ! **Variables explode with the instance size**
> 10,000 SKUs, 1,000,000 orders and 2 warehouses already lead to $2 \times 10^{10}$ binary variables $Y_{m,i,k}$!

### Implementation Challenges

- Small instance with 10 SKUs and 1000 customer orders
- **CPLEX 20.1.0** needs 3100 seconds to solve the problem
- Computation time scales exponentially
- $\rightarrow$ **Not applicable** in real world applications!

---

Any idea what

could be done?

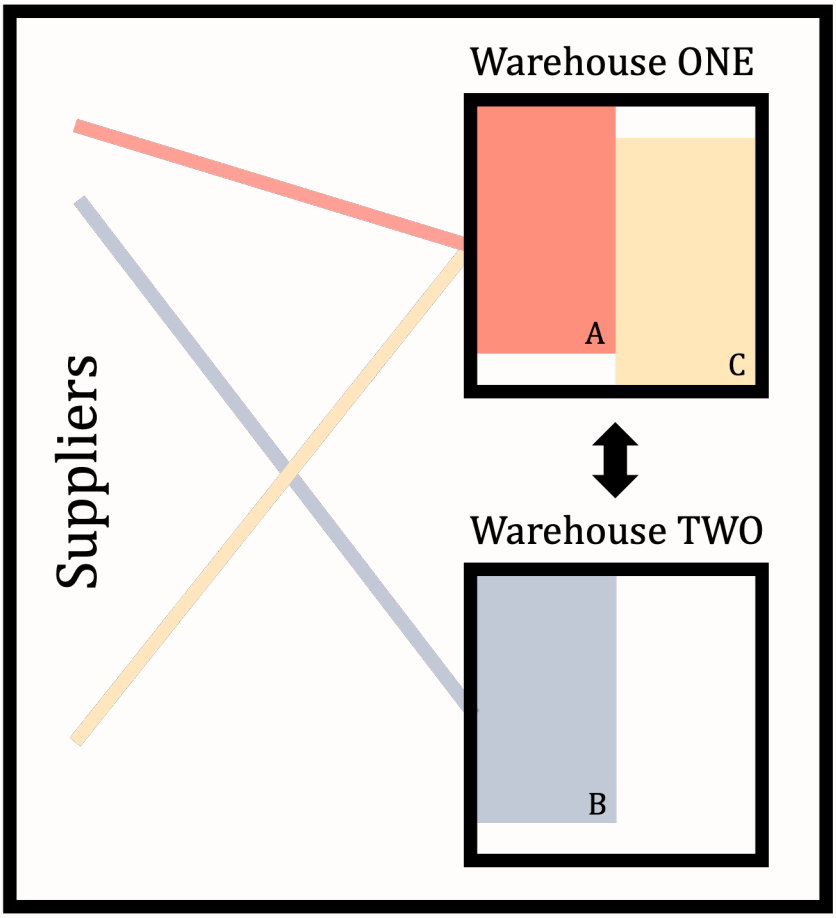
## Structure - Version 2

### Heuristic Approach

- **Heuristic**: Fast, but not necessarily optimal
- **Approximation**: Not guaranteed to be optimal, but close
- **Computational Effort**: Reasonable even for large instances

> **Different view on the problem**
> Focus on the warehouses and the coappearance of SKUs! Discard the exact information about the customer orders.

## Objective




**Question: What could be the objective?**

Maximize the coappearance of products that are often **part of the same customer orders**.

## Transaction Matrix

```
T = [
    1 1 1 0;
    1 1 1 0;
    1 1 0 0;
    1 0 0 1;
    1 0 0 1;
    1 0 0 1;
    1 0 0 1;
    0 0 1 1
]

# Create the coappearance matrix
Q = T' * T
```

```
println("Coappearance matrix Q:")
display(Q)
```

```
Coappearance matrix Q:
```

```
4×4 Matrix{Int64}:
 7  3  2  4
 3  3  2  0
 2  2  3  1
 4  0  1  5
```

## Coappearance Matrix

- $Q$ is a **symmetric matrix**
- Proposed by A. Catalán and M. Fisher [4]
- $Q = T^\top T$ where $Q = (q_{i,j})_{i,j \in \{1,\dots,|\mathcal{I}|\}}$
- $q_{i,j}$ shows how often $i$ and $j$ appear **in the same order**

**Question: What do the principal diagonal values tell us?**

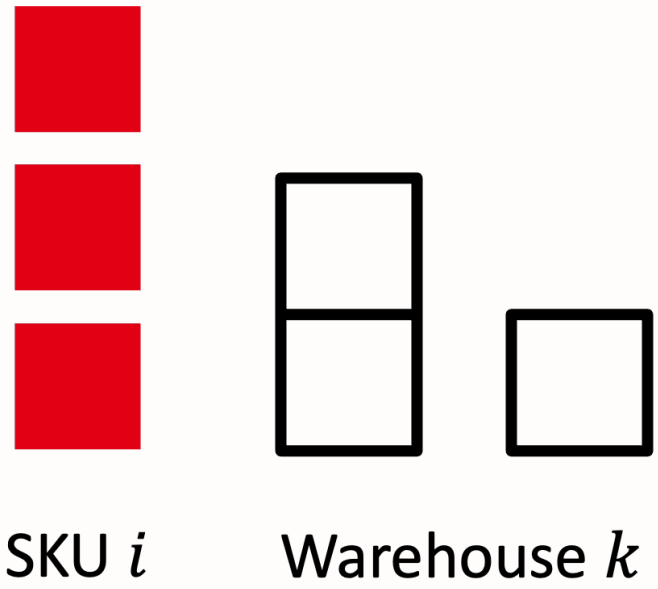
- In how many **orders** each SKU appeared
- As $t_{m,i}$ is binary, $q_{i,i}$ counts orders, **not units sold**

## How to approach the problem?

- **Greedy Heuristic**[3]: Allocation based on matrix
- **Mathematical Model**[4]: Maximizes coappearance
- **GRASP**[5]: Good on small instances
- **New**: Max. coappearance with non-linear solver
- **New**: Heuristic based on Chi-Square Tests

[3] Simple and very fast, A. Catalán and M. Fisher [4]
[4] Computationally intensive with CPLEX, S. Zhu, X. Hu, K. Huang, and Y. Yuan [5]
[5] Greedy Randomized Adaptive Search Procedure, S. Zhu, X. Hu, K. Huang, and Y. Yuan [5]

## Basic Setting




## Available Data (Version 2)

**Question: What could be the sets?**

- $\mathcal{I}$ - Set of products indexed by $i \in \{1, 2, ..., |\mathcal{I}|\}$
- $\mathcal{K}$ - Set of warehouses indexed by $k \in \{1, ..., |\mathcal{K}|\}$

> ! **No customer order information is needed!**
> We can focus on the SKUs and the warehouses, making the problem **much smaller**!

## Available Parameters

**Question: What are possible parameters?**

- $c_k$ - Storage space of warehouse $k \in \{1, ..., |\mathcal{K}|\}$, measured in number of SKUs
- $\boldsymbol{Q} = (q_{i,j})_{i,j \in \{1,...,|\mathcal{I}|\}}$ - Coappearance matrix

> ! **Transactional Data replaced**
> Instead of the transactional data, we just **use the coappearance matrix** in our model!

# Model Formulation

## Decision Variables?

> **We have the following sets:**
> - $\mathcal{I}$ - Set of products indexed by $i \in \{1, 2, ..., |\mathcal{I}|\}$
> - $\mathcal{K}$ - Set of warehouses indexed by $k \in \{1, ..., |\mathcal{K}|\}$

> **Our objective is to:**
> Maximize the coappearance of products that are often part of the same customer orders. **In more mathematical terms:** Maximize the sum of all unique pair-wise values $q_{i,j}$ of all SKUs stored in the same warehouse.

**Question: What could be our decision variable/s?**

## Decision Variables

- $X_{i,k}$ - 1, if SKU $i \in \mathcal{I}$ is stored in $k \in \mathcal{K}$, 0 otherwise

> **Only one variable per SKU and warehouse!**
> As we don't need the customer order information, we only need to make a decision for each SKU and warehouse pair!

## Decision Variable in Julia

**Question: How could we formulate the variable in Julia?**

```
using JuMP, SCIP # SCIP is a non-commercial solver for quadratic models

warehouses = ["Hamburg", "Berlin"] # Add warehouses as a vector
skus = ["A", "B", "C", "D"] # The four SKUs from our transactional data

warehouse_model = Model(SCIP.Optimizer)


@variable(warehouse_model, X[i in skus, k in warehouses], Bin)
```

```
2-dimensional DenseAxisArray{JuMP.VariableRef,2,...} with index sets:
    Dimension 1, ["A", "B", "C", "D"]
    Dimension 2, ["Hamburg", "Berlin"]
And data, a 4×2 Matrix{JuMP.VariableRef}:
 X[A,Hamburg]  X[A,Berlin]
 X[B,Hamburg]  X[B,Berlin]
 X[C,Hamburg]  X[C,Berlin]
 X[D,Hamburg]  X[D,Berlin]
```

## Objective Function

> **We need the following:**
> - $X_{i,k}$ - 1, if SKU $i \in \mathcal{I}$ is stored in $k \in \mathcal{K}$, 0 otherwise
> - $q_{i,j}$ - Coappearance of SKU $i \in \mathcal{I}$ and $j \in \mathcal{I}$

> **Our objective is to:**
> Maximize the sum of all unique pair-wise values $q_{i,j}$ of all SKUs stored in the same warehouse. Note, that this is a **quadratic objective function**!

**Question: What could the objective function look like?**

## Quadratic Objective Function

$$\text{Maximize} \quad \sum_{i=2}^{|\mathcal{I}|} \sum_{j=1}^{i-1} \sum_{k \in \mathcal{K}} X_{i,k} \times X_{j,k} \times q_{i,j}$$

> This is a **quadratic objective function**!
> The quadratic terms are $X_{i,k} \times X_{j,k}$. This objective function is based on the **Quadratic Multiple Knapsack Problem (QMKP)**, formulated by A. Hiley and B. A. Julstrom [6].

## Objective Function in Julia

**Question: How could we formulate this in Julia?**

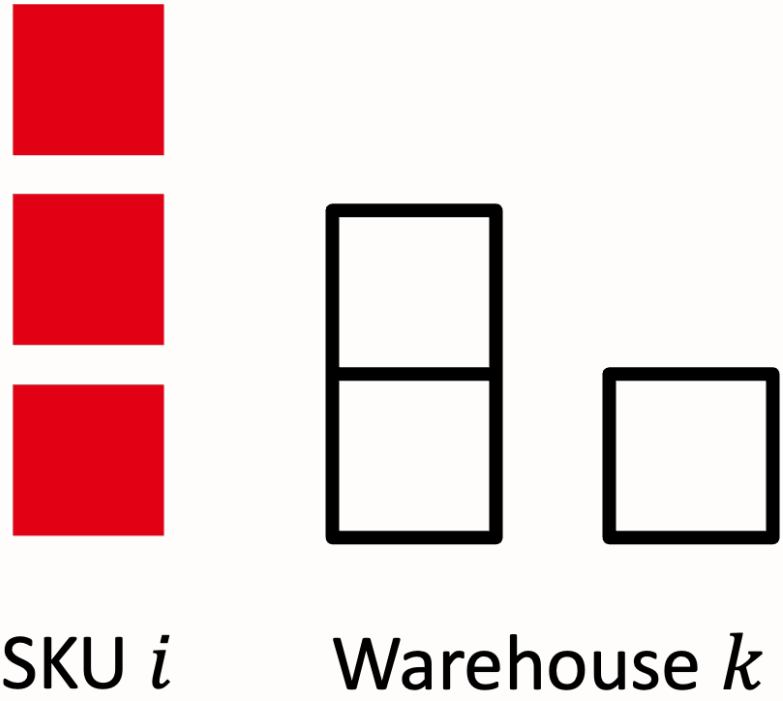
```
# We reuse the matrix Q = T' * T computed from our transactional data

@objective(warehouse_model,
    Max,
    sum(
        X[skus[i], warehouses[k]] * X[skus[j], warehouses[k]] * Q[i,j]
        for i in 2:length(skus)
        for j in 1:i-1
        for k in 1:length(warehouses)
    )
)
```

```
3 X[B,Hamburg]*X[A,Hamburg] + 3 X[B,Berlin]*X[A,Berlin] + 2
X[C,Hamburg]*X[A,Hamburg] + 2 X[C,Berlin]*X[A,Berlin] + 2
X[C,Hamburg]*X[B,Hamburg] + 2 X[C,Berlin]*X[B,Berlin] + 4
X[D,Hamburg]*X[A,Hamburg] + 4 X[D,Berlin]*X[A,Berlin] +
X[D,Hamburg]*X[C,Hamburg] + X[D,Berlin]*X[C,Berlin]
```

# Constraints

## What constraints?




**Question: What constraints?**

- Allocate each SKU **at least once**
- Warehouses have a **finite capacity**
- Capacity is **not exceeded**

## Single Allocation Constraint?

> ! **The goal of this constraint is to:**
> Ensure that each SKU is allocated at least once.

> i **We need the following variable:**
> - $X_{i,k}$ - 1, if SKU $i \in \mathcal{I}$ is stored in $k \in \mathcal{K}$, 0 otherwise

**Question: What could the constraint look like?**

## Single Allocation Constraint

$$\sum_{k \in \mathcal{K}} X_{i,k} \geq 1 \quad \forall i \in \mathcal{I}$$

> **Remember, this is the variable:**
> - $X_{i,k}$ - 1, if SKU $i \in \mathcal{I}$ is stored in $k \in \mathcal{K}$, 0 otherwise

**Question: How could we change the constraint to ensure that each SKU is allocated only once?**

**Question: How could we add the constraint in Julia?**

## Single Allocation in Julia

```
@constraint(warehouse_model, single_allocation[i in skus],
    sum(X[i, k] for k in warehouses) >= 1
)
```

```
1-dimensional DenseAxisArray{JuMP.ConstraintRef{JuMP.Model,
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
MathOptInterface.GreaterThan{Float64}}, JuMP.ScalarShape},1,...} with index
sets:
    Dimension 1, ["A", "B", "C", "D"]
And data, a 4-element Vector{JuMP.ConstraintRef{JuMP.Model,
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
MathOptInterface.GreaterThan{Float64}}, JuMP.ScalarShape}}:
 single_allocation[A] : X[A,Hamburg] + X[A,Berlin] ≥ 1
 single_allocation[B] : X[B,Hamburg] + X[B,Berlin] ≥ 1
 single_allocation[C] : X[C,Hamburg] + X[C,Berlin] ≥ 1
 single_allocation[D] : X[D,Hamburg] + X[D,Berlin] ≥ 1
```

## Capacity Constraints?

> **The goal of these constraints is to:**
> Ensure that the capacity of each warehouse is not exceeded.

> **We need the following variables and parameters:**
> - $X_{i,k}$ - 1, if SKU $i \in \mathcal{I}$ is stored in $k \in \mathcal{K}$, 0 otherwise
> - $c_k$ - Storage space of warehouse $k \in \mathcal{K}$

**Question: What could the second constraint be?**

## Capacity Constraints

$$\sum_{i \in \mathcal{I}} X_{i,k} \leq c_k \quad \forall k \in \mathcal{K}$$

**And that's basically it!**

**Question: How could we add the second constraint in Julia?**

## Capacity Constraints in Julia

```
capacities = Dict("Hamburg" => 2, "Berlin" => 2) # Add capacities

@constraint(warehouse_model, capacity[k in warehouses],
    sum(X[i, k] for i in skus) <= capacities[k]
)
```

```
1-dimensional DenseAxisArray{JuMP.ConstraintRef{JuMP.Model,
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
MathOptInterface.LessThan{Float64}}, JuMP.ScalarShape},1,...} with index sets:
    Dimension 1, ["Hamburg", "Berlin"]
And data, a 2-element Vector{JuMP.ConstraintRef{JuMP.Model,
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
MathOptInterface.LessThan{Float64}}, JuMP.ScalarShape}}:
 capacity[Hamburg] : X[A,Hamburg] + X[B,Hamburg] + X[C,Hamburg] + X[D,Hamburg]
≤ 2
 capacity[Berlin] : X[A,Berlin] + X[B,Berlin] + X[C,Berlin] + X[D,Berlin] ≤ 2
```

## QMK Model

$$\text{Maximize} \quad \sum_{i=2}^{|\mathcal{I}|} \sum_{j=1}^{i-1} \sum_{k \in \mathcal{K}} X_{i,k} \times X_{j,k} \times q_{i,j}$$

subject to:

$$\sum_{k \in \mathcal{K}} X_{i,k} \geq 1 \ \forall i \in \mathcal{I}$$

$$\sum_{i \in \mathcal{I}} X_{i,k} \leq c_k \forall k \in \mathcal{K}$$

$$X_{i,k} \in \{0,1\} \ \forall i \in \mathcal{I}, k \in \mathcal{K}$$

## QMK Model in Julia

```
set_attribute(warehouse_model, "display/verblevel", 0) # Hide solver output
optimize!(warehouse_model)

println("The optimal objective value is: ", objective_value(warehouse_model))
println("The optimal solution is: ", value.(X))
```

```
The optimal objective value is: 6.0
The optimal solution is: 2-dimensional DenseAxisArray{Float64,2,...} with index
sets:
    Dimension 1, ["A", "B", "C", "D"]
    Dimension 2, ["Hamburg", "Berlin"]
```

```
And data, a 4×2 Matrix{Float64}:
 1.0  0.0
 0.0  1.0
 0.0  1.0
 1.0  0.0
```

**Question: What does this value tell us?**

### Interpreting the Solution

- Total capacity $(2 + 2)$ **equals** the number of SKUs
- Hence, each SKU is stored **exactly once**
- Optimal: **A and D** share a warehouse, so do **B and C**

> **The objective value is 6**
> It sums the coappearances of SKUs sharing a warehouse: $q_{A,D} + q_{B,C} = 4 + 2 = 6$. It does **not** directly count the avoided split orders!

## Model Characteristics

### Characteristics

- Is the model formulation **linear/ non-linear?**
- What kind of **variable domain** do we have?
- Do we know the **split-orders** based on the **objective value?**
- Why **couldn't we use HiGHS** as solver?

### Choosing a solver

- Identify **problem structure**, e.g. LP, MIP, NLP, MIQP, MIQCP, ...
- What is the **size** of the problem?
- Is a **commercial** solver needed?

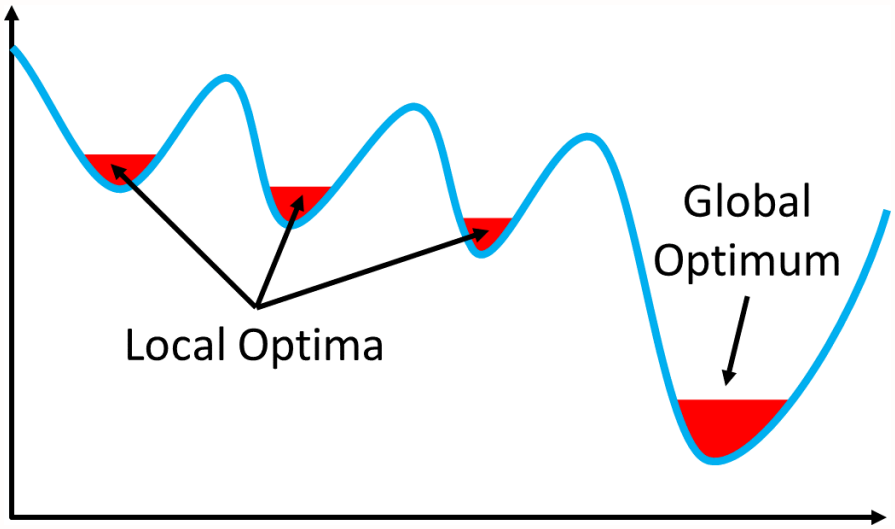
> **Commercial Solvers**
> Commercial solvers are **faster** and **more robust** than open source solvers but also **more expensive**. During your studies, you can use most of them for free though! Nonetheless, we will only use open source solvers in this course.

### Our Problem Class

- Objective: **quadratic** terms $X_{i,k} \times X_{j,k}$
- Constraints: all **linear**
- Variables: all **binary**

> **Mixed-Integer Quadratic Program (MIQP)**
> Our model is a **MIQP**, as only the objective is quadratic. It is not a MIQCP, which would also allow quadratic **constraints**. MIQCP solvers, e.g. SCIP, subsume MIQPs and can thus solve our model.

## Global vs Local Optimality




## Solver Comparison[6]

| SKUs | SCIP | HiGHS | Gurobi |
|---|---|---|---|
| 100 | 118s | X | ~400s |
| 1,000 | 1,011s (18%) | X | ~200s (2%) |
| 10,000 | X | X | X |

## Solver Comparison: Takeaways

- **SCIP**: Best open-source for MIQP/MIQCP, limited scalability
- **Commercial solvers**: Better but still fail on 10,000+ SKUs
- **Conclusion**: Heuristics necessary for realistic problems!

## Model Assumptions

**Questions: On model assumptions**

- What assumptions have we made?
- Problem with allocating SKUs to multiple warehouses?

[6] Based on numerical experiments with QMK instances. Times are averages over the instances solved to optimality; the percentage shows the share of instances solved within one hour, X = none solved within one hour.

- What else might pose a problem in the real world?

> **Duplicates can inflate the objective**
> With slack capacity, the model may store one SKU in **several warehouses**. A shared pair then counts **once per warehouse**, raising the objective without avoiding any further splits.

## Model Limitations

- **Stock-outs**: Model assumes **perfect availability**
  - Real stockouts create splits despite optimal allocation
- **Storage costs**: No differentiation between costs
- **Demand dynamics**: Weather, promotions, seasonality
- **SKU sizes**: Uniform capacity assumption unrealistic
- **Shipping costs**: No consideration of distance or weight

## Practical Limitations

**Allocation changes**:

- Frequent reallocation **disrupts operations**
- Physical inventory movements costly
- Need for change minimization

**Workload imbalance**:

- Optimal allocation may **overload** some warehouses
- High-frequency SKUs cluster together
- Trade-off: splits vs. balance

# Impact

Can this be

applied?

## Problem Size is Crucial

- Up to 1,000 SKUs → **commercial solvers**
- More than 1,000 SKUs → **heuristics**
- For example, the **CHI** heuristic

> **CHI-Heuristic**
> Detect dependencies between products and allocate them accordingly, as products within orders can have dependencies and products are bought with different frequencies!

# CHI Heuristic

## Statistical Independence

**Question: When are two SKUs independent?**

If purchasing one doesn't affect purchasing the other:

$$P(A \cap B) = P(A) \times P(B)$$

**Example**: If $A$ appears in 10% of orders and $B$ in 20%:

- **Independent**: Appear together in $0.10 \times 0.20 = 2\%$ of orders
- **Dependent**: Appear together in **5% of orders** → customers buy them together!

## Chi-Square Test Intuition

Use **chi-square tests** to detect if SKUs are dependent:

- Calculate **expected coappearances** under independence
- Compare with **actual coappearances**
- If difference is significant → SKUs are **dependent**
- Allocate dependent SKUs to **same warehouse**!

## From Tests to Allocations

> **Two-Phase Approach**
> **Phase 1**: Build allocation using dependencies
> **Phase 2**: Refine with local search

**Result:** Works on large instances, 50,000 SKUs in 10 minutes!

# In Practice

## Real-World Challenges

**Question: What challenges might arise in practice?**

- **New products**: Allocate SKUs without historical data?
- **Dynamic inventory**: SKUs appear and disappear
- **Different sizes**: SKUs consume varying storage space
- **Computational limits**: Problems with 50,000+ SKUs

# Case: A Large Online Retailer

**Question:** Which of 100,000+ SKUs goes to which warehouse?

## Rolling Horizon Approach

In the following case study [7], we:

- used a **5-week training window** for allocation decisions
- updated allocations **weekly** based on recent patterns
- balanced **stability** vs. **responsiveness**

> **Tip**
> In the case study, SKUs were grouped by **category-brand combinations** to reduce the problem size while maintaining allocation quality. New SKUs inherit the allocation of their cluster!

## Case Study

**Problem characteristics**:

- 100,000+ SKUs
- Several million orders
- Multiple warehouses
- Storage capacity constraints

**Heuristics compared**[7]:

- **CHI**: Chi-Square tests [7]
- **GP, GO, GS, BS**: Greedy [4]
- **RA**: Random allocation

## Implementation Results

**Retailer's status quo**: 6.95% of all orders were split

**Theoretical reduction of these splits**:

- CHI: **82.25%** reduction
- BS: **62.63%** reduction
- GS: **59.16%** reduction

## Conclusion

- Splits are **of no benefit**, except **faster customer deliveries**
- **Increase workload, packaging and shipping costs**
- Mathematical Optimization of **"full" problem not solvable**
- **CHI** Heuristic close to mathematical optimization

[7] QMKP is not applicable for instance in case study

> **And that's it for today's lecture!**
> We now have covered the Quadratic Multiple Knapsack Problem and are ready to start solving some tasks in the upcoming tutorial.

Questions?

---

# Literature

References for this lecture:

- A. Catalán and M. Fisher [4]: Integer model and greedy heuristics
- A. Hiley and B. A. Julstrom [6]: The Quadratic Multiple Knapsack Problem
- S. Zhu, X. Hu, K. Huang, and Y. Yuan [5]: Mathematical model and GRASP
- T. Vlćek and G. Voigt [7]: CHI heuristic and case study

For more interesting literature to learn more about Julia, take a look at the literature list of this course.

[1] Pitney Bowes Inc., "Pitney Bowes Parcel Shipping Index Reveals 48 Percent Growth in Parcel Volume since 2014." Accessed: May 27, 2021. [Online]. Available: https://www.businesswire.com/news/home/20170830005628/en/Pitney-Bowes-Parcel-Shipping-Index-Reveals-48

[2] Pitney Bowes Inc., "Pitney Bowes Parcel Shipping Index Reports Continued Growth Bolstered by China and Emerging Markets." Accessed: May 27, 2021. [Online]. Available: https://www.businesswire.com/news/home/20191010005148/en/

[3] Pitney Bowes Inc., "Pitney Bowes Parcel Shipping Index Reports Continued Growth as Global Parcel Volume Exceeds 100 billion for First Time Ever." Accessed: May 27, 2021. [Online]. Available: https://www.businesswire.com/news/home/20201012005150/en/

[4] A. Catalán and M. Fisher, "Assortment Allocation to Distribution Centers to Minimize Split Customer Orders," *SSRN Electronic Journal*, vol. , no. , p. , 2012, doi: 10.2139/ssrn.2166687.

[5] S. Zhu, X. Hu, K. Huang, and Y. Yuan, "Optimization of product category allocation in multiple warehouses to minimize splitting of online supermarket customer orders," *European Journal of Operational Research*, vol. 290, no. 2, pp. 556–571, 2021, doi: 10.1016/j.ejor.2020.08.024.

[6] A. Hiley and B. A. Julstrom, "The Quadratic Multiple Knapsack Problem and Three Heuristic Approaches to It," in *Proceedings of the 8th Annual Conference on Genetic and Evolutionary Computation*, M. Keijzer, Ed., New York, NY: Association for Computing Machinery, 2006, pp. 547–552. doi: 10.1145/1143997.1144096.

[7] T. Vlćek and G. Voigt, “Optimizing SKU-Warehouse Allocations to minimize Split Parcels in E-Commerce Environments,” Working paper, to be submitted to Manufacturing & Service Operations Management, 2024.